

IT HAPPENS @ RAALE #2

Data In, Answers Out

Build a CSV → Kafka → Neo4j Chatbot Pipeline



3 Hours Build & Report · 5:00 PM - 8:00 PM

2 hours 30 minutes build | 30 minutes report

STUDENT HANDOUT

PART 0

What This Project Actually Is

0.1 The problem in plain language

A software company receives spreadsheets all the time — exports from a CRM, dumps from a partner system, a support tool's nightly download. Somebody has to load that data somewhere useful, and then somebody else has to answer questions about it: "how many rows are in the Billing group?", "which customer is linked to which order?", "show me everything connected to this account."

Right now this is manual. A person opens the CSV, imports it into some system by hand, and answers questions about it by scrolling and squinting, or by writing a query themselves.

Your job: build a pipeline that takes a CSV in the front door and lets a person ask questions about it in plain English out the back door — with nothing manual in between.

Rows and columns in a spreadsheet are secretly a network: a row references another row, a value in one column repeats across many rows and quietly links them together. A graph database is built for exactly that shape — nodes and relationships, not rows and columns. That is why the data lands in Neo4j and not just a table. Kafka is the conveyor belt between the upload and the database, so the part that accepts a file and the part that writes to the graph never have to trust each other's speed.

0.2 What makes this different

Most student database exercises end like this: open a script, run it once, watch five hundred rows get inserted, close the terminal. That is homework. Nobody outside that room can use it, and it only runs when you press Run.

This project does not end there. You must deliver:

- **A running service.** Someone must be able to open a browser, drag in a CSV they have never shown you before, and get a working chatbot on top of it — automatically, with nobody from your team present.
- **Something another team can start on their own machine,** with one command, without phoning you for help. This is why the project uses Docker.
- **Proof that it works.** You must answer "does the chatbot actually know what it's talking about?" with evidence, not a feeling.

There are two classic ways to fail this, and both are common in real companies:

A pipeline with no chatbot is a filing cabinet nobody can query — the data went in, and it is now somebody's problem to get it back out.

A chatbot with no real data behind it is a puppet. It will produce a confident, plausible-sounding sentence whether or not anything in your database supports it. That is worse than useless — it does damage, quietly, because people believe it.

You are marked on the whole path, front door to back door.

PART 1

Important: The Chatbot Must Be Honest

⚠ READ THIS SECTION CAREFULLY

Teams lose marks every time by getting this wrong.

Your chatbot can be as simple as matching a question to a Cypher query template, or, if your team wants and your organisers allow it, calling a language model to turn the user's English question into a Cypher query, or to phrase the final sentence nicely. Either approach is acceptable.

What is not acceptable is a chatbot that answers from general knowledge instead of your graph. If the question cannot be answered from what is actually sitting in Neo4j, the correct answer is "I don't have that in the data" — not a plausible-sounding guess.

Why this matters

- **Trust** — a wrong-but-confident answer is worse than no answer, because people act on it.

- **Verifiability** — a judge (or a real user) must be able to check your chatbot's claim against the graph, not just take its word for it.
- **It costs nothing to admit uncertainty** — an honest "I don't know" is one line of code; a hallucinated answer is a support ticket next week.
- **The point of the exercise** — wiring an upload form to an LLM chat window is a five-minute task. Building something that retrieves real data and grounds an answer in it is the skill being taught.

If you use a language model anywhere in the chatbot, you must return the actual Cypher query you ran and the raw result alongside the phrased answer — see the contract in Part 4. A confident sentence with nothing behind it scores zero on the grounding component.

Ask your organisers whether an LLM or API key is permitted at this event before assuming you can call one.

PART 2

Before the Session

You will not be given time to pull large Docker images during the hackathon. Arrive ready.

2.1 Install and verify Docker

Install Docker Desktop (Windows / Mac) or Docker Engine (Linux). Then actually run something — do not just check that it is installed:

```
docker run --rm hello-world
docker compose version
```

If either command fails, fix it before the session. There is no recovery time in the schedule.

2.2 Pre-pull the heavy images

Kafka and Neo4j images are large. Pull them at home, on a good connection:

```
docker pull apache/kafka:3.7.0
docker pull neo4j:5.24-community
docker pull python:3.11-slim
```

2.3 Bring test CSVs

The task is a "dynamic CSV" upload — your pipeline must accept any CSV, not one you trained against in advance. Prepare three files at home so you are not scrambling on the night:

- A **small, clean CSV** (10–20 rows) to sanity-check the pipe end to end.
- A **larger CSV** (thousands of rows) to see how your loader behaves under real volume.

- A **deliberately broken CSV** — missing header row, ragged columns, stray commas — to test that your service fails politely instead of crashing.

2.4 Your Neo4j credentials

These are fixed for the event. Wire them in as environment variables in your compose file — never hard-code them into your source:

```
Database Name: CSV_Graph_DB
Password:      csvgraphdb
```

2.5 A stack that fits the clock

You may build this in any language. Below is one combination known to finish inside the time. Deviate only if your team is genuinely stronger elsewhere — and say so in your report.

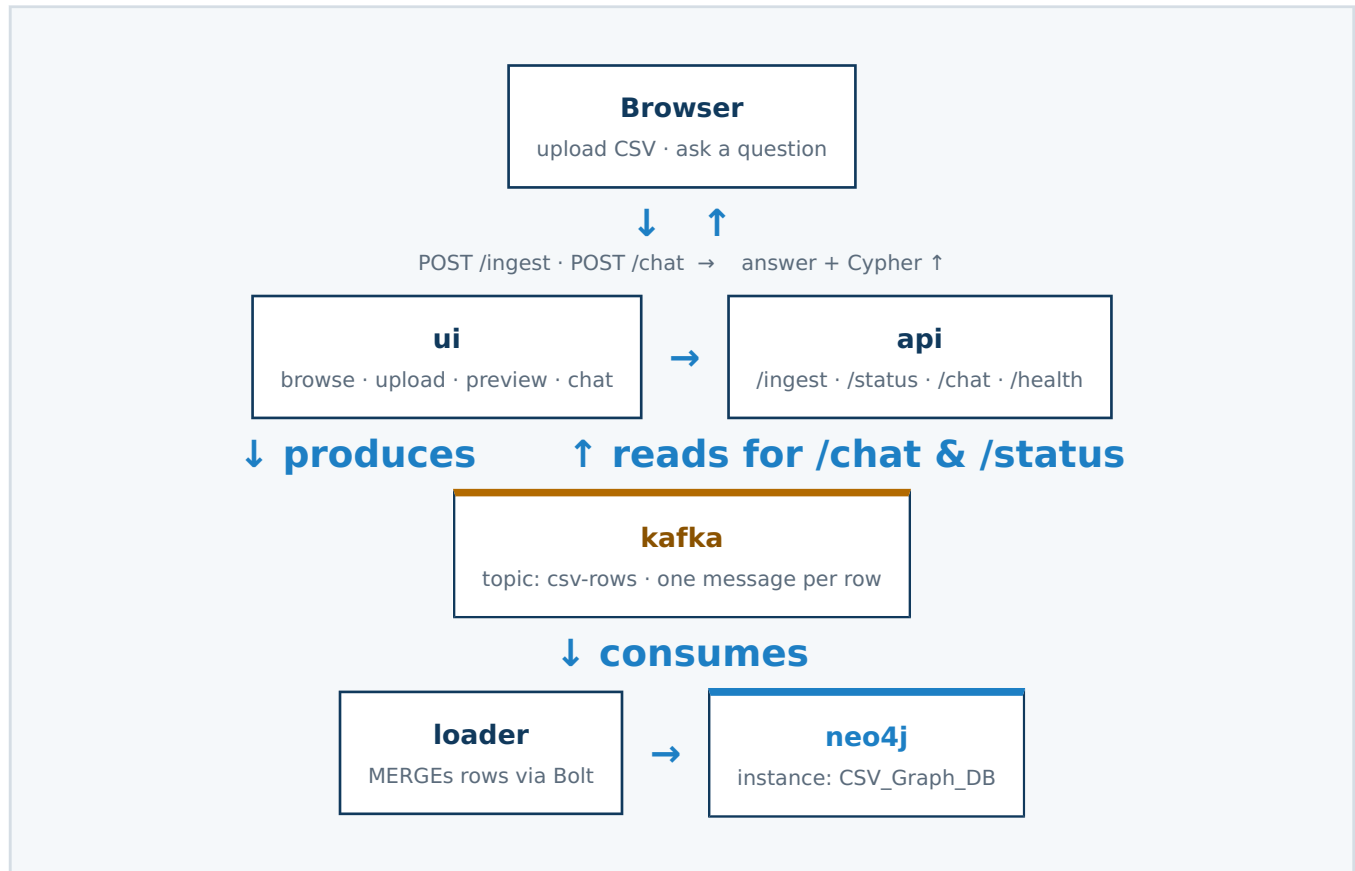
Layer	Suggested	Note
Language	Python 3.11 or Node.js 18	Fewest moving parts; both have a mature Neo4j driver and Kafka client
UI	A single lightweight page (plain HTML + fetch, or React)	A working ugly UI beats a broken pretty one
Broker	Apache Kafka, single broker, KRaft mode	No separate ZooKeeper container needed on modern images
Graph DB	Neo4j 5.x, Community edition, official image	Ships with Cypher and a browser UI at :7474 for free
Driver	neo4j-driver (JS) or the official Python driver	Do not hand-roll Bolt or HTTP calls to Neo4j
API	FastAPI or Flask, or Express	Validates request bodies for you; async support helps with Kafka
Chatbot	Question → Cypher template map, or an LLM call if permitted	Always return the Cypher and the raw result with the answer

Three notes worth knowing before you start:

- Kafka needs a few seconds to elect itself leader even as a single broker. Your producer and consumer must retry on connection-refused, not crash.
- Neo4j accepts the Bolt port later than the container reports "started." Treat this the same way you treat API readiness — see 6.3.
- `pip install --no-cache-dir` / `npm ci --omit=dev` save real space for free; keep the trainer-equivalent (loader) and API images minimal.

What You Are Building

Five moving parts, controlled by one `docker-compose.yml` file.



- (1) **ui** — lets a user browse for a CSV, upload it, preview the rows that were received, and open a chat box once loading is under way.
- (2) **api** — one backend, three jobs: accept an upload and publish it to Kafka (`POST /ingest`), report load progress (`GET /status`), and answer questions (`POST /chat`).
- (3) **kafka** — a single-broker topic. One message per CSV row. This decouples "accepting the file" from "writing it to the graph," so a slow or briefly-down database never causes a failed upload.
- (4) **loader** — consumes the topic and MERGES each row into Neo4j as it arrives, so status can report real progress while the file is still loading.
- (5) **neo4j** — the graph itself, instance `CSV_Graph_DB`, queried by both the loader (writes) and the api's `/chat` endpoint (reads).

Why put Kafka in the middle instead of writing straight to Neo4j from the upload handler?

- The upload handler returns immediately instead of blocking on however long the whole file takes to write.
- If Neo4j is briefly unreachable, messages queue safely in Kafka instead of being dropped on the floor.
- You can replay the topic to reload the graph without asking the user to upload the file again.

- This is exactly how ingestion pipelines are built at real companies — producers and consumers are deployed, scaled, and restarted independently.

PART 4

The Interface Contract

Your API must accept and return these shapes. Other teams' tests and the judges' checks depend on it.

UPLOAD / INGEST

```
POST /ingest
Content-Type: multipart/form-data (field name: file)
-> 202 Accepted
{
  "job_id": "b3f1",
  "rows_received": 1000,
  "status": "queued"
}
```

STATUS

```
GET /status?job_id=b3f1
{
  "job_id": "b3f1",
  "status": "loading",    // queued | loading | complete | failed
  "rows_total": 1000,
  "rows_loaded": 640,
  "rows_failed": 3
}
```

HEALTH CHECK

```
GET /health
{
  "status": "ok",
  "kafka_connected": true,
  "neo4j_connected": true
}
```

`/health` must report anything other than `ok` until both Kafka and Neo4j are genuinely reachable — not merely "the container has started."

CHAT

```
POST /chat
{ "question": "How many rows belong to the Billing group?" }
-> 200 OK
{
  "answer": "There are 128 rows where group = 'Billing'.",
  "cypher": "MATCH (r:Row {group: 'Billing'}) RETURN count(r)",
  "result": [{"count(r)": 128}],
  "grounded": true
}
```

If the graph has no answer, `grounded` must be `false` and `answer` must say so plainly — never fabricate a number. See Part 1.

PART 5

The Idempotent Load Rule (Mandatory)

Because the CSV is dynamic — you do not know its columns ahead of time — keep the graph model generic until you have time to enrich it:

```
(:Dataset {id, filename, uploaded_at})
-[:HAS_ROW]->
(:Row {row_index, col_1, col_2, ... })
```

Every row must be written with `MERGE`, keyed on a stable identifier such as `dataset_id + row_index` — never `CREATE`. If your loader uses `CREATE`, restarting docker compose or replaying the Kafka topic will duplicate every row.

⚠ NO PARTIAL CREDIT ON THIS ONE

Two clean runs against the same CSV must produce the same node and relationship counts. Producing duplicate nodes on a second load scores zero on the reproducibility component of Part 11 — no partial credit.

If you subsample or transform rows before writing them, do it inside the loader — never mutate the file in `./data`.

PART 6

How to Actually Do This

Read this whole part before coding. It exists so you do not lose an hour to a trap we already know about.

6.1 Keep the graph model boring

Dataset → Row, one property per column, is enough to pass. You are marked on the system, not on a beautifully normalised schema. If you finish early, then enrich the model — detect that a column looks like a foreign key and turn it into a real relationship — but keep the generic model working as your fallback.

6.2 Build the pipe before the logic

This is the single most important paragraph in the document. The most common way to fail is to have a clever chatbot at 7:40 and no working compose file. Build the plumbing with fake logic first, then fill it in.

So in your first block of time, build the skeleton with fake logic:

- **ui** uploads any file and shows "uploaded"
- **api**'s `/ingest` accepts the file and writes one dummy message to Kafka, then returns a fake `job_id`
- **loader** consumes that one message and MERGEs a single dummy node into Neo4j, then logs done
- **api**'s `/status` returns a hardcoded "complete"
- **api**'s `/chat` returns a hardcoded canned answer with `grounded: false`

Once `docker compose up` runs all five end to end, replace the dummies one at a time. From that moment onward, every minute you spend improves something that already works — and if you run out of time, you still have a working system.

6.3 "Started" is not "ready"

`depends_on` in Docker Compose only waits for a container to start, not for Kafka to elect a leader or Neo4j to accept Bolt connections.

Fix it properly: add healthchecks to kafka and neo4j, and make api and loader depend on them with `condition: service_healthy`. A retry loop inside your own connection code is an acceptable alternative — but say in your report which you chose and why.

6.4 Keep credentials out of the image

The Neo4j password from Part 2.4 goes into environment variables in `docker-compose.yml` (or a `.env` file it reads), never into a Dockerfile line or a hard-coded string in your source. Be ready to explain why: an image with a baked-in password can leak the password anywhere the image is pushed, and rotating the password means rebuilding the image.

6.5 Status will lie to you if you let it

It is tempting to mark a job complete once every Kafka message has been consumed. That is not the same as every row being safely in Neo4j — a write can fail. Track `rows_failed` honestly, and do not report complete until loaded + failed accounts for every row that was received.

6.6 Make it repeatable

Fix any identifiers your loader generates so a rerun on the same file produces the same `dataset_id`. Judges will run `docker compose down -v` followed by `docker compose up` on your test CSV and compare row counts against your report. They must match.

6.7 Assume hostile input

Real services receive garbage. Before the freeze, throw these at your API and make sure it responds sensibly instead of crashing:

- an empty CSV file
- a CSV with a header row but zero data rows
- a file that is not a CSV at all
- a chat question with no relevant data in the graph
- a chat question sent before any CSV has been uploaded

A clean error message is a pass. A stack trace, a dead container, or a confidently wrong chat answer is not.

PART 7

Requirements

7.1 Must-have — this is your pass

#	Requirement
1	<code>docker compose up</code> on a clean machine brings up ui, api, kafka and neo4j, and the loader starts consuming — with zero manual steps
2	The uploaded CSV reaches Neo4j only via the Kafka topic, never written directly from the upload handler
3	Every row is written with <code>MERGE</code> on a stable key — no duplicates on a second load of the same file
4	<code>GET /health</code> reports not-ok until Kafka and Neo4j are genuinely reachable
5	<code>GET /status</code> reports queued, loading, complete or failed with real row counts, not a hardcoded value
6	<code>POST /chat</code> follows the contract in Part 4, including cypher and result on every answer
7	A question with no supporting data in the graph returns <code>grounded: false</code> and says so, instead of guessing
8	All base images pinned to a version — no <code>latest</code> anywhere
9	Containers run as a non-root user

#	Requirement
10	Two clean runs against the same CSV produce identical row and relationship counts

7.2 Stretch — this is how you win

- api image under 400 MB
- p95 `/ingest` response time under 200 ms even for a 10,000-row file, actually measured, with the method stated
- The ui shows live progress while a large file is still loading, not just a spinner
- Survives every item in the hostile-input list in 6.7
- The chatbot detects likely foreign-key-style columns and turns them into real relationships instead of flat properties
- A second `docker compose up` on the same file skips re-publishing rows that are already MERGED
- Multi-stage builds, so build tools never ship in the runtime image

PART 8 Timeline

Time	Phase	You are done when
5:00 – 5:10	Read, agree the architecture, split the work	Everyone knows what they own
5:10 – 5:35	Skeleton — five services, dummy logic end to end	<code>docker compose up</code> runs ui → api → kafka → loader → neo4j
5:35 – 6:00	Real ui — upload, preview, chat box	A real CSV can be dragged in and its rows previewed
6:00 – 6:30	Real ingest + loader — rows really reach Neo4j via Kafka	A MATCH query in Neo4j Browser shows real rows
6:30 – 7:00	Real api — <code>/status</code> reports true progress and <code>/health</code> is honest	<code>curl</code> on <code>/status</code> matches what's actually in the graph
7:00 – 7:20	Real chatbot — grounded answers with Cypher shown	A real question about the uploaded file gets a correct, grounded answer
7:20 – 7:25	Hardening — non-root, pinned tags, hostile input	The Part 10 checklist passes
7:25 – 7:30	BUILD FREEZE — commit and push	Nothing further is edited
7:30 – 8:00	Report	REPORT.md committed

Nominate one person as timekeeper and have them call out each checkpoint aloud.

🕒 THE 7:25 FREEZE IS HARD

Work that is not committed does not exist.

PART 9

The Report (30 minutes, 30 marks)

One file: `REPORT.md`, in your repository. Plain language. These sections, in this order.

9.1 What we built

Five sentences plus the architecture. State honestly what works and what does not. An accurate account of a half-working pipeline scores far above an optimistic account of the same pipeline.

9.2 The data and the graph model

The CSVs you actually tested with, row counts, and the exact node labels, relationship types and properties your loader produces. If you enriched the generic Dataset/Row model, explain how you detected the enrichment and what you did when detection was wrong.

9.3 Methods

Fill this in — one line per row:

Decision	Chosen	Rejected	Reason
Ingest path			
Idempotency key			
Chatbot approach			
How api knows kafka/ neo4j are ready			

We are checking whether you chose or merely defaulted.

9.4 Results

A table of at least eight questions you actually asked your chatbot, whether the answer was correct and grounded, and — most important — your explanation of why the ones that failed, failed. That paragraph carries more marks than the table above it.

Question asked	Answer given	Correct?	Grounded?

Question asked	Answer given	Correct?	Grounded?

9.5 How we worked

Who owned what, and whether that changed. Planned versus actual at each checkpoint in Part 8. Then two decisions, in this form:

Decision: what you decided — Options considered: what else was on the table — Chosen because: your reasoning — Cost accepted: what it cost you — Would revisit if: what would change your mind

And one dead end — something you tried, when you abandoned it, and what told you to stop. Knowing when to stop is a marked skill.

9.6 Limitations and next steps

What breaks in production. Be specific. "Add monitoring" earns nothing. "We have no way to detect that a re-uploaded file with the same name but different content should replace, not merge with, the old dataset" earns a lot.

9.7 How to run it

Exact commands. A judge must be able to run your project from this section alone, with no verbal help from you.

PART 10

Pre-Freeze Checklist

At 7:15, stop adding features and verify every line.

- `docker compose down -v` then `docker compose up` works from clean
- No `latest` tag anywhere
- Neo4j password comes from an environment variable, not hard-coded
- Containers run as non-root
- `/health` reports not-ok before Kafka and Neo4j are genuinely reachable
- `/status` reports real row counts, including failures, not a hardcoded value
- API survives an empty CSV, a non-CSV file, and a chat question asked before any upload
- A second load of the same CSV does not duplicate nodes
- Chat answers include the Cypher query and result, and say so honestly when ungrounded
- `REPORT.md` committed
- Everything pushed

PART 11

Marking

Component	
docker compose up works clean, first time, no manual steps	15
Correct pipeline architecture — Kafka decouples ingest from load	10
Healthcheck and correct startup ordering (Kafka, Neo4j, loader, api)	8
Container hygiene — pinned tags, non-root, credentials via env, sane image sizes	7
UI works end to end — upload, preview, and chat all function in the browser	5
API correctness and input handling	10
Idempotent load — reruns do not duplicate data	10
Chatbot groundedness — answers only from the graph, honestly says "I don't know"	10
Report: methods and decisions	12
Report: results interpretation	8
Report: process and honesty	5
Total	100

Read that distribution carefully. Chatbot cleverness is worth 10 marks. Engineering and reasoning are worth 90.

A team with a template-only chatbot, a clean pipeline and a sharp report will beat a team with an LLM-powered chatbot and a broken compose file. That is not an accident — it is the point of the exercise.

PART 12

Frequently Asked

Do we need internet during the hackathon?

Only if you didn't pre-pull the Kafka and Neo4j images. Do that beforehand — see Part 2.2.

Do we need a fixed dataset ahead of time?

No. The task is a dynamic CSV upload. Bring your own test files — see Part 2.3.

Can we use any programming language?

Yes. Python or Node is the path of least resistance because of mature Kafka and Neo4j clients, but nothing in the requirements demands either.

Do we need multiple Kafka brokers or a separate ZooKeeper container?

No. A single broker in KRaft mode is enough and boots faster.

Can our chatbot call a language model?

Ask your organisers for the rule in force at this event. If permitted, the model may only turn the question into a Cypher query or phrase the final sentence — it must not answer from general knowledge. See Part 1.

What if the loader crashes partway through a large file?

`/status` must report failed and the row count actually reached. A silent crash that leaves `/status` saying loading forever is treated as a failed run.

Can we skip the ui and just prove it with curl?

Yes, if you're short on time — a working ingest → kafka → neo4j → chat pipeline with an honest report beats a broken five-container system with a pretty front end every time.

Our chatbot gets a lot of questions wrong. Are we finished?

No. Chatbot correctness is only 10 of 100 marks. A chatbot that honestly admits what it doesn't know, sitting on a clean and well-evaluated pipeline, scores well. Report the weak results honestly — that explanation earns marks.

Good luck — and build the pipe first.